Acoustic Feature Extraction for Audio Deepfake Detection

Aditi Paul

Biometrics Assignment

Dataset: ASVspoof Logical Access (LA) corpus 2019

1. Abstract

Audio deepfakes threaten voice-based biometric systems, and most detectors still start from handcrafted acoustic features. This report builds a feature extraction and analysis pipeline for separating genuine (bona fide) speech from spoofed speech. From each recording we compute LFCC, MFCC, a CQCC-like cepstrum, spectral centroid, bandwidth and roll-off, zero crossing rate (ZCR) and RMS energy, and summarise each with mean, standard deviation, skewness and kurtosis, giving 250 features per utterance. We compare the feature distributions of the two classes, rank features with Welch's t-test (Benjamini-Hochberg FDR corrected) and Cohen's *d*, and check how useful the features are by training two detectors: a HistGradientBoosting (HGB) classifier on the summary statistics and a Light CNN (LCNN) on 2D LFCC + delta + delta-delta tensors.

The strongest single discriminators are higher-order statistics of the cepstra (for example the skewness of MFCC 8, Cohen's $d = 0.92$, and the standard deviation of CQCC-like coefficient 4, $d = 0.87$), not simple energy or zero-crossing statistics. On the development partition HGB reaches an EER of 2.21% and the LCNN 1.12%. On the evaluation partition, which contains unseen attacks, the EER rises to 9.32% (HGB) and 6.38% (LCNN). The LCNN is better in every comparison, mainly because it makes far fewer false alarms on genuine speech.

2. Description

2.1 Problem and motivation

Spoofing countermeasures (CMs) decide whether an utterance comes from a live human or from text-to-speech (TTS), voice conversion (VC) or replay. Which acoustic representation the CM sees largely determines how well it works. The task is to extract

several complementary features, compare their behaviour on genuine and spoofed speech, and identify which separate the classes best.

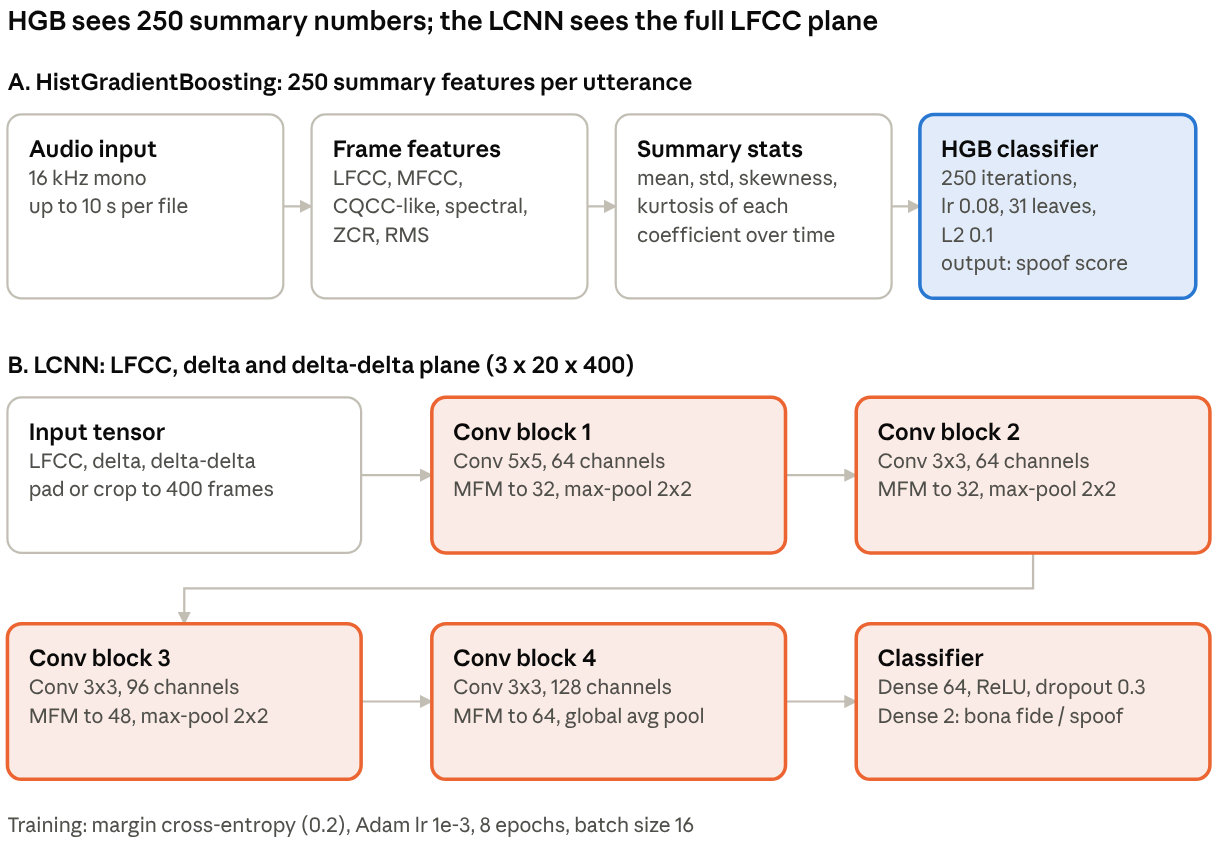
2.2 Pipeline overview

1. **Preprocessing.** Load FLAC at 16 kHz, downmix to mono, cap at 10 s, parse the five-column protocol files and map labels to 0 (bona fide) and 1 (spoof).
2. **Frame-level features.** STFT with 512-point FFT and 10 ms hop (160 samples), then:
   - **LFCC:** 20 linearly spaced triangular filters, log energy, DCT-II.
   - **MFCC:** 20 coefficients (librosa).
   - **CQCC-like:** 84-bin constant-Q transform, log magnitude, DCT, first 20 coefficients.
   - **Spectral descriptors:** centroid, bandwidth, roll-off, ZCR, RMS.
3. **Utterance-level summary.** Mean, standard deviation, skewness and excess kurtosis per coefficient across time. Spectral descriptors use mean and standard deviation only. This gives $3 \times 20 \times 4 + 5 \times 2 = 250$ features.
4. **Statistical analysis.** Welch's t-test and Cohen's *d* per feature, BH-FDR correction at $\alpha = 0.05$, and KDE plots for the top-ranked features.
5. **Validation by classification.** HGB on the 250 summary features, and an LCNN on a $3 \times 20 \times 400$ LFCC/Δ/ΔΔ tensor. Metrics are accuracy, precision, recall, F1, false positive rate (genuine flagged as spoof), false negative rate (spoof accepted as genuine), ROC-AUC and approximate EER.

2.3 Models

HistGradientBoosting: 250 iterations, learning rate 0.08, 31 leaf nodes, L2 = 0.1, seed 42.

LCNN: four convolutional blocks (5×5 then 3×3 kernels) with Max-Feature-Map (MFM) activation, max-pooling and global average pooling, then a 64-unit dense layer with 0.3 dropout. It is trained with a margin cross-entropy loss (margin 0.2) and Adam (lr 1e-3, weight decay 1e-5) for 8 epochs with batch size 16.



architecture of both models · HGB pipeline and LCNN

Figure A. Architecture of both detectors. The HGB works on 250 utterance-level statistics, while the LCNN reads the whole LFCC time-frequency plane with its deltas. MFM is Max-Feature-Map, which keeps the larger of two halves of the channels.

3. Datasets Used

ASVspoof 2019 Logical Access (LA) is the corpus on which the ASVspoof 2021 LA task is built. The 2021 LA evaluation re-uses the 2019 LA evaluation attacks, transmitted over telephony channels, and the 2021 task trains on 2019 LA data. The assignment names ASVspoof 2021, but the experiments here use the clean 2019 LA partitions due to memory limitations in my personal computer. Channel and codec effects from 2021 were not evaluated (see Future Work).

Partition	Protocol rows	Bona fide	Spoof	Files used for features
Train	25,380	2,580	22,800	21,844 (2,219 bona fide / 19,625 spoof)
Dev	24,844	2,548	22,296	21,843

Partition	Protocol rows	Bona fide	Spoof	Files used for features
Eval	71,237	7,355	63,882	21,844

Train and dev contain attacks A01 to A06. The evaluation set adds unseen attack systems, which is what makes it a test of generalisation.

Important caveats. (i) The local dataset mirror was incomplete: 3,536 train, 3,001 dev and 49,393 eval protocol entries had no audio file and were skipped, so only a subset of each partition was processed. The evaluation results therefore cover about 31% of the official evaluation list. (ii) The data are heavily imbalanced (about 10% bona fide), so plain accuracy is misleading. EER and FPR are reported alongside it. (iii) One dev file (LA_D_8413012) failed to load and was skipped by the LCNN pipeline.

4. Implementation Code

The full notebook (`ASVspoof2019_LA_full_pipeline.ipynb`) was run on Google Colab with a CUDA GPU. The key components are reproduced below.

Configuration and audio loading

```
SR, N_FFT, HOP_LENGTH = 16000, 512, 160
N_MFCC = N_LFCC = 20
MAX_AUDIO_SECONDS, MAX_LFCC_FRAMES = 10, 400

def load_audio(path, sr=SR):
    y, _ = librosa.load(path, sr=sr, mono=True, duration=MAX_AUDIO_SECONDS)
    if y.size == 0:
        raise ValueError(f'Empty audio: {path}')
    return y.astype(np.float32)
```

Feature extractors

```
from scipy.fftpack import dct

def _filterbank(y, sr, n_filters=20, scale='mel', n_fft=N_FFT,
hop=HOP_LENGTH):
    S = np.abs(librosa.stft(y, n_fft=n_fft, hop_length=hop)) ** 2
    if scale == 'mel':
        fb = librosa.filters.mel(sr=sr, n_fft=n_fft, n_mels=n_filters)
    else:  # linear-frequency triangular filters (LFCC)
        freqs = np.linspace(0, sr / 2, n_fft // 2 + 1)
```

```
        centers = np.linspace(0, sr / 2, n_filters + 2)
        fb = np.zeros((n_filters, len(freqs)), dtype=np.float32)
        for i in range(n_filters):
            left, center, right = centers[i:i+3]
            fb[i] = np.maximum(fb[i], (freqs-left)/(center-left))
            fb[i] = np.minimum(fb[i], (right-freqs)/(right-center))
            fb[i] = np.clip(fb[i], 0, 1)
    return np.log(np.maximum(fb @ S, 1e-10))

def extract_lfcc(y, sr=SR, n_lfcc=N_LFCC):
    return dct(_filterbank(y, sr, n_lfcc, 'linear'), type=2, axis=0,
norm='ortho')

def extract_mfcc(y, sr=SR, n_mfcc=N_MFCC):
    return librosa.feature.mfcc(y=y, sr=sr, n_mfcc=n_mfcc, n_fft=N_FFT,
hop_length=HOP_LENGTH)

def extract_cqcc_like(y, sr=SR, n_coeffs=20):
    C = np.abs(librosa.cqt(y, sr=sr, hop_length=HOP_LENGTH, n_bins=84,
bins_per_octave=12))
    return dct(np.log(np.maximum(C, 1e-8)), type=2, axis=0, norm='ortho')
[:n_coeffs]

def extract_spectral(y, sr=SR):
    return {
        'spectral_centroid':  librosa.feature.spectral_centroid(y=y, sr=sr,
n_fft=N_FFT, hop_length=HOP_LENGTH)[0],
        'spectral_bandwidth': librosa.feature.spectral_bandwidth(y=y, sr=sr,
n_fft=N_FFT, hop_length=HOP_LENGTH)[0],
        'spectral_rolloff':   librosa.feature.spectral_rolloff(y=y, sr=sr,
n_fft=N_FFT, hop_length=HOP_LENGTH)[0],
        'zero_crossing_rate': librosa.feature.zero_crossing_rate(y,
hop_length=HOP_LENGTH)[0],
        'rms':                librosa.feature.rms(y=y, frame_length=N_FFT,
hop_length=HOP_LENGTH)[0],
    }
```

Utterance-level summarisation

```
def summarize_matrix(matrix, prefix):
    feats = {}
    for i, row in enumerate(matrix):
        r = np.asarray(row, dtype=np.float64); r = r[np.isfinite(r)]
        vals = [r.mean(), r.std(), skew(r), kurtosis(r)] if r.size > 3 else
[0.0] * 4
        for stat, v in zip(['mean', 'std', 'skew', 'kurtosis'], vals):
            feats[f'{prefix}_{i}_{stat}'] = float(v)
    return feats
```

Statistical analysis (Welch t-test, Cohen's d, BH-FDR)

```
def cohens_d(x, y):
    pooled = ((len(x)-1)*np.var(x, ddof=1) + (len(y)-1)*np.var(y, ddof=1)) /
(len(x)+len(y)-2)
    return 0.0 if pooled <= 0 else (np.mean(y) - np.mean(x)) /
np.sqrt(pooled)

def analyze_features(df):
    bona, spoof = df[df.label == 0], df[df.label == 1]
    rows = []
    for col in [c for c in df.select_dtypes(np.number) if c != 'label']:
        x, y = bona[col].dropna().values, spoof[col].dropna().values
        t, p = ttest_ind(x, y, equal_var=False)
        d = cohens_d(x, y)
        rows.append(dict(feature=col, mean_bonafide=x.mean(),
mean_spoof=y.mean(),
                         t_statistic=t, p_value=p, cohens_d=d,
abs_cohens_d=abs(d)))
    return pd.DataFrame(rows).sort_values('abs_cohens_d', ascending=False)

def benjamini_hochberg(p):
    p = np.asarray(p, float); n = len(p); order = np.argsort(p)
    adj = np.minimum.accumulate((p[order] * n / np.arange(1, n + 1))[::-1])
[::-1]
    out = np.empty(n); out[order] = np.clip(adj, 0, 1)
    return out
```

Baseline classifier and security metrics

```
hgb = HistGradientBoostingClassifier(max_iter=250, learning_rate=0.08,
        max_leaf_nodes=31, l2_regularization=0.1,
random_state=42).fit(X_train, y_train)

def calculate_metrics(y_true, y_pred, y_score):
    tn, fp, fn, tp = confusion_matrix(y_true, y_pred, labels=[0, 1]).ravel()
    fpr, tpr, _ = roc_curve(y_true, y_score); fnr = 1 - tpr
    idx = np.nanargmin(np.abs(fpr - fnr))
    return dict(accuracy=accuracy_score(y_true, y_pred),
f1_spoof=f1_score(y_true, y_pred),
                FPR=fp/(fp+tn), FNR=fn/(fn+tp),
                roc_auc=roc_auc_score(y_true, y_score),
                eer_approx=(fpr[idx] + fnr[idx]) / 2)
```

LCNN input tensor, model and loss

```
def prepare_lfcc_tensor(path):
    lfcc = extract_lfcc(load_audio(path))
    stack = np.stack([lfcc, librosa.feature.delta(lfcc, order=1),
                      librosa.feature.delta(lfcc,
order=2)]).astype(np.float32)
    T = stack.shape[-1]
    return (np.pad(stack, ((0,0),(0,0),(0, MAX_LFCC_FRAMES - T))) if T <
MAX_LFCC_FRAMES
            else stack[:, :, :MAX_LFCC_FRAMES])

class MaxFeatureMap2D(nn.Module):
    def forward(self, x):
        a, b = torch.chunk(x, 2, dim=1); return torch.maximum(a, b)

class LCNN(nn.Module):
    def __init__(self, num_classes=2):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(3, 64, 5, padding=2),   MaxFeatureMap2D(),
nn.MaxPool2d(2),
            nn.Conv2d(32, 64, 3, padding=1),  MaxFeatureMap2D(),
nn.MaxPool2d(2),
            nn.Conv2d(32, 96, 3, padding=1),  MaxFeatureMap2D(),
nn.MaxPool2d(2),
            nn.Conv2d(48, 128, 3, padding=1), MaxFeatureMap2D(),
nn.AdaptiveAvgPool2d((1, 1)))
        self.classifier = nn.Sequential(nn.Flatten(), nn.Linear(64, 64),
```

```
nn.ReLU(),
                                        nn.Dropout(0.3), nn.Linear(64,
num_classes))
    def forward(self, x): return self.classifier(self.features(x))

class MarginCrossEntropy(nn.Module):
    def __init__(self, margin=0.2): super().__init__(); self.margin = margin
    def forward(self, logits, target):
        adj = logits.clone()
        adj[torch.arange(logits.size(0)), target] -= self.margin
        return nn.functional.cross_entropy(adj, target)
```

The training loop runs 8 epochs with Adam over a `DataLoader` that uses a `safe_collate_fn` to drop unreadable files. The seed is 42 for Python, NumPy and PyTorch.

5. Results with Graphs and Analysis of Results

5.1 Feature-level statistical analysis

All 250 features were tested between 2,219 bona fide and 19,625 spoof training utterances. With samples this large, p-values underflow to 0 for the top features (t-statistics of roughly −50 to −60), so **statistical significance does not discriminate between features; effect size does.** Cohen's *d* is therefore used for ranking.

Rank	Feature	Mean (bona fide)	Mean (spoof)	Cohen's *d*
1	MFCC 8 skewness	−0.659	−0.201	0.916
2	CQCC-like 4 std	2.278	2.702	0.871
3	MFCC 3 std	26.39	32.29	0.837
4	CQCC-like 3 std	3.313	3.935	0.829
5	MFCC 9 skewness	−0.711	−0.298	n/a (top 20)
6	Spectral bandwidth mean (Hz)	1689.4	1552.6	n/a (top 20)

The top-20 list also includes MFCC 7 mean (−6.90 vs −11.96), MFCC 8 mean, MFCC 2 std, MFCC 10 skewness, LFCC 0 skewness, LFCC 1 std, LFCC 16 to 18 means and several CQCC-like skewness and std terms. The notebook saves the full ranked table with BH-

adjusted p-values to `feature_statistical_analysis.csv`. KDE plots for the six highest-ranked features were produced in the notebook (Section 14). Their separation matches the mean shifts above: the two class densities overlap substantially but are offset.

Reading the results

- **Cepstral shape statistics carry the signal.** Skewness and standard deviation of mid-order MFCC and CQCC-like coefficients dominate the ranking. Spoofed speech has a more variable cepstral envelope (larger std in MFCC 3 and CQCC-like 3 and 4) and a different temporal asymmetry (less negative skew). This is consistent with vocoder and synthesis artefacts that alter spectral-envelope dynamics.
- **Spectral bandwidth** is the only simple spectral descriptor in the top six: spoofed audio is slightly narrower in bandwidth (about 137 Hz lower on average), which suggests band-limited or smoothed high-frequency content.
- **ZCR, RMS energy and spectral centroid** do not appear in the top 20 by $|d|$. Used alone they are weaker discriminators than the cepstral statistics. Overall energy is also largely controlled by recording level and normalisation, which carries little spoofing information.
- **All three cepstral families contribute.** MFCC dominates the top ranks, with LFCC and CQCC-like features also present. The LFCC entries are mostly high-order means (16 to 18), which fits the usual finding that LFCC retains useful high-frequency detail that the mel scale compresses.
- **No single feature is enough.** The largest $|d|$ is about 0.9, a large effect but with substantial class overlap. Reliable detection needs many features together, which is what the classifiers exploit.

5.2 Classification performance

Model	Partition	Accuracy	F1 (spoof)	FPR (genuine rejected)	EER
HistGradientBoosting	Dev	98.03%	98.91%	16.90%	2.21%
LCNN	Dev	99.17%	99.54%	7.14%	1.12%
HistGradientBoosting	Eval	90.89%	94.71%	9.97%	9.32%
LCNN	Eval	91.54%	95.07%	3.91%	6.38%

Other values: ROC-AUC on dev is 0.9972 (HGB) and 0.9991 (LCNN). On eval it is 0.9672 (HGB) and 0.9791 (LCNN). Spoof miss rate (FNR) on eval is 9.02% (HGB) and 8.99% (LCNN). LCNN training loss fell from 0.198 to 0.020 and training accuracy rose from 93.6%

to 99.5% over 8 epochs.

Analysis

- **The LCNN beats the HGB baseline everywhere.** Relative to HGB it cuts the false-positive rate by 2.37× on dev and 2.55× on eval, and the EER by 1.98× on dev and 1.46× on eval. Bona fide recall on dev rises from 0.83 to 0.93.
- **Accuracy hides the main weakness.** With about 90% spoof data, a model that always says "spoof" scores about 90% accuracy. The HGB's 98% dev accuracy still hides rejecting 1 in 6 genuine speakers. In a biometric setting that false-rejection rate matters a great deal, so FPR and EER are the more informative figures.
- **Large generalisation gap.** EER rises roughly 4× for HGB (2.2% to 9.3%) and 5.7× for LCNN (1.1% to 6.4%) from dev to eval. This is expected because the evaluation set contains attack algorithms never seen during training. Handcrafted global statistics and a small CNN both overfit to the artefacts of A01 to A06.
- **Why the LCNN helps.** The LCNN sees the full time-frequency LFCC plane and its deltas, so it can use local spectro-temporal artefacts that are lost when each coefficient is collapsed into four global statistics.
- **Eval FNR is similar for both models (about 9%).** Neither detector reliably catches the unseen attacks. The LCNN's gain on eval comes mainly from fewer false rejections of genuine speech.

Figures. Figures 1 to 3 below are the plots produced by the notebook.

Figure 1. KDE of the six top-ranked features, bona fide (0, blue) vs spoof (1, orange). The spoof curves are shifted and wider for the cepstral std and skewness features, and shifted to lower values for spectral bandwidth, with clear overlap in every panel.

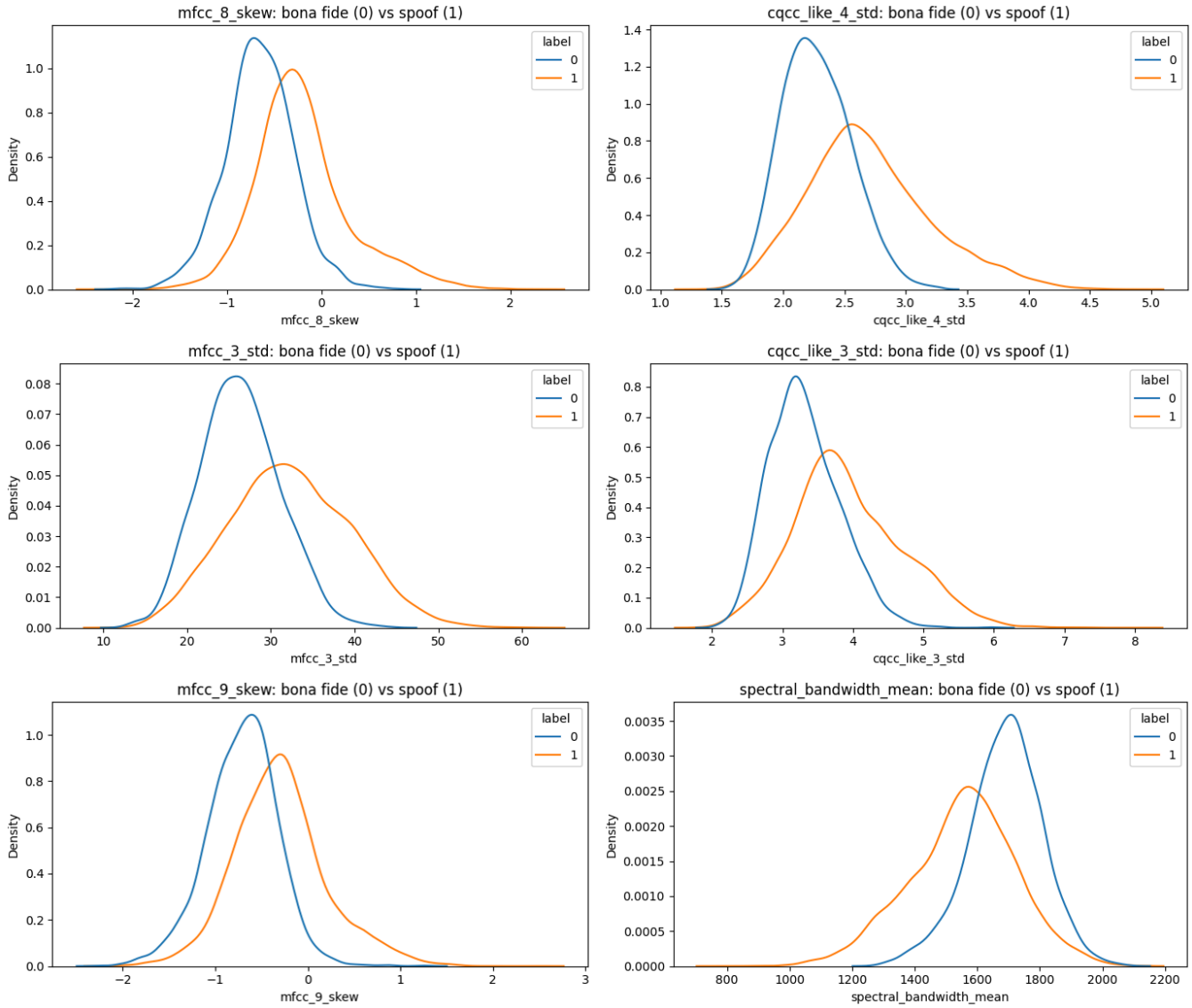


Figure 2. Dev-set confusion matrices for HistGradientBoosting (left) and the LCNN (right). Both rarely miss a spoof; the LCNN rejects far fewer genuine utterances.

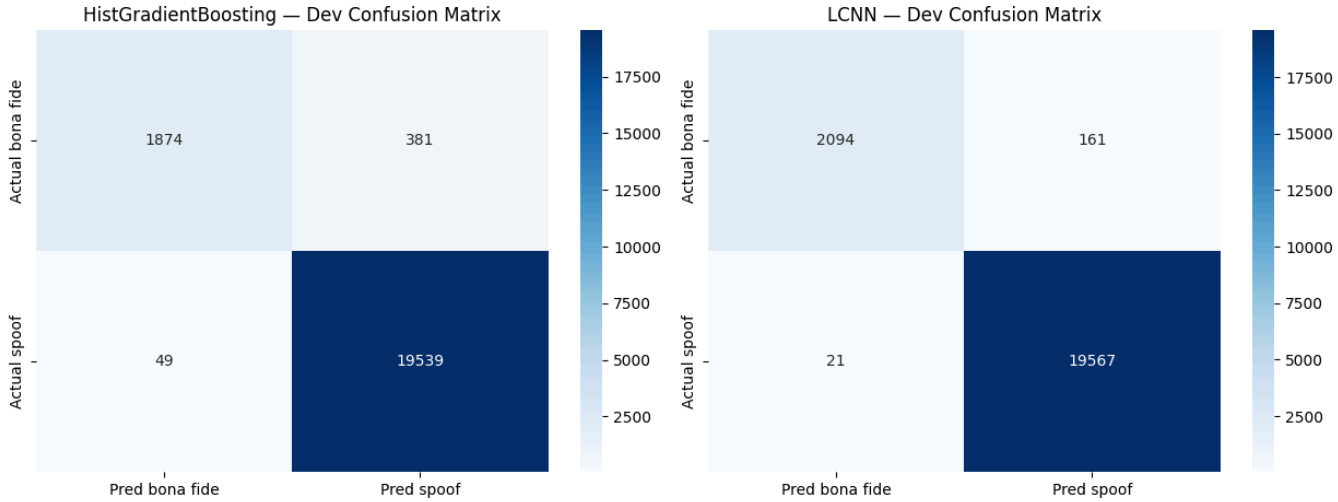
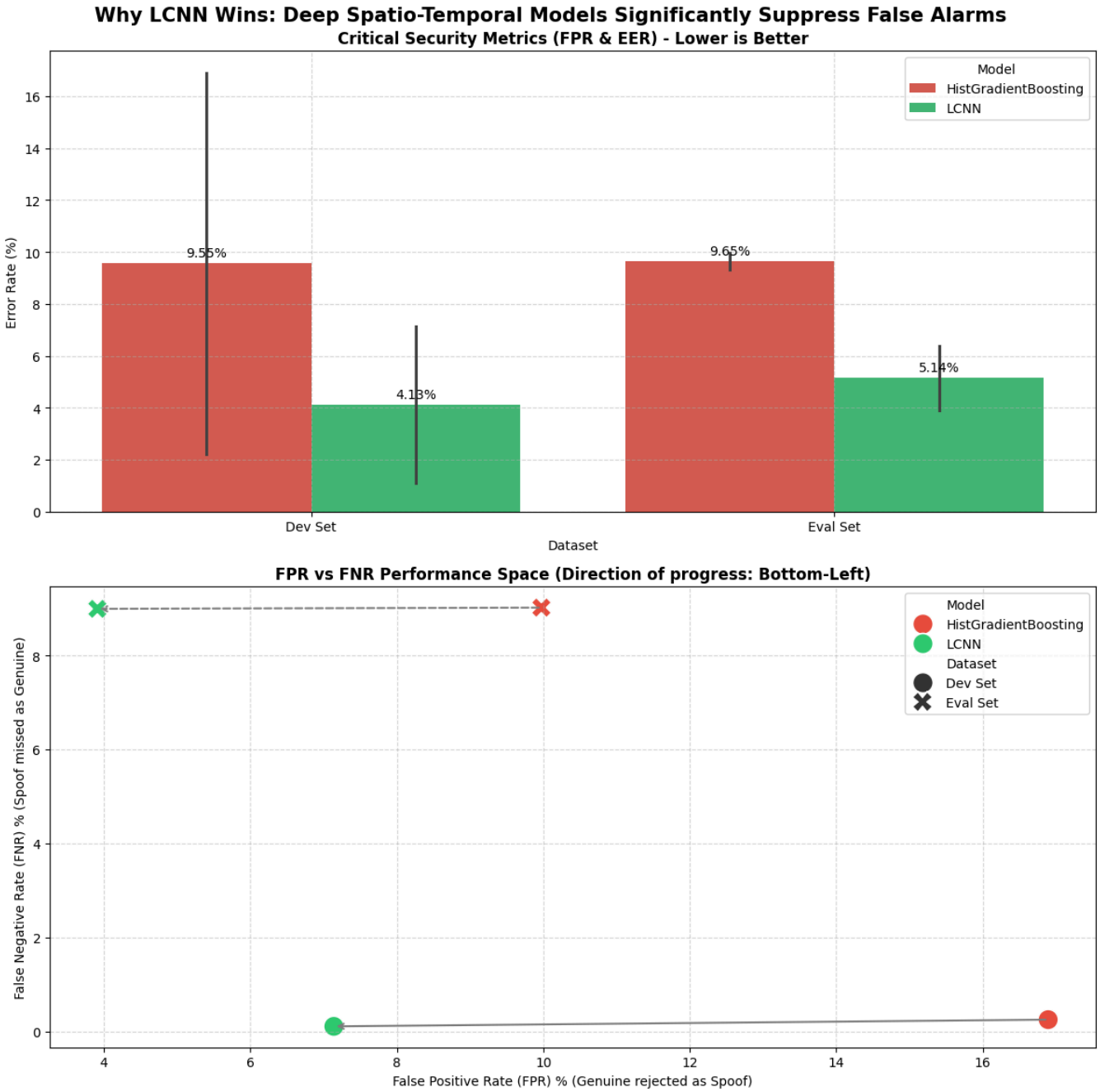


Figure 3. FPR and EER by model and partition (top), and the FPR-vs-FNR plane with arrows from HGB to LCNN (bottom). Lower and to the left is better.



5.3 Limitations of this study

- The dataset is ASVspoof 2019 LA, not the 2021 channel-degraded data, and an incomplete mirror was used, so only a subset of each partition was processed. EERs here are **not** directly comparable to official challenge numbers.
- Spectral flatness, listed in the assignment, was **not** implemented in the notebook. Spectral bandwidth and roll-off were computed instead.

- The CQCC-like feature is a log-CQT followed by a DCT. It omits the uniform resampling step of true CQCC, so it approximates the published feature.
- EER is approximated from the ROC curve (the point where FPR is closest to FNR), and the LCNN was trained for a fixed 8 epochs with no validation-based model selection. Results are from a single run (seed 42) with no confidence intervals.

6. Future Work

1. **Complete the feature set.** Add spectral flatness and a faithful CQCC with uniform resampling, then rerun the effect-size ranking.
2. **Use the full data.** Process all train, dev and eval files, and the ASVspoof 2021 LA/DF evaluation sets, to measure robustness to telephony codecs and compression.
3. **Per-attack analysis.** Break EER and feature effect sizes down by attack ID (A01 to A19) to see which features fail on which synthesis methods.
4. **Better statistics.** Add non-parametric tests (Mann-Whitney U), ROC-AUC per feature, mutual information and permutation importance from the HGB, and report confidence intervals over several seeds.
5. **Better generalisation.** Try data augmentation (codec, noise, RIR), longer or random-crop inputs, feature fusion across LFCC/CQCC/MFCC, and self-supervised front-ends such as wav2vec 2.0 or AASIST-style models.
6. **Metrics.** Report the tandem detection cost function (t-DCF) together with EER, and calibrate scores for deployment thresholds.

7. References

1. X. Wang et al., "ASVspoof 2019: A large-scale public database of synthesized, converted and replayed speech," *Computer Speech & Language*, vol. 64, 2020.
2. M. Todisco et al., "ASVspoof 2019: Future horizons in spoofed and fake audio detection," *Proc. Interspeech*, 2019.
3. J. Yamagishi et al., "ASVspoof 2021: Accelerating progress in spoofed and deepfake speech detection," *ASVspoof 2021 Workshop*, 2021. See also X. Liu et al., "ASVspoof 2021: Towards spoofed and deepfake speech detection in the wild," *IEEE/ACM Trans. Audio, Speech, Lang. Process.*, vol. 31, 2023.
4. M. Todisco, H. Delgado and N. Evans, "Constant Q cepstral coefficients: A spoofing countermeasure for automatic speaker verification," *Computer Speech & Language*, vol. 45, 2017.

5. M. Sahidullah, T. Kinnunen and C. Hanilçi, "A comparison of features for synthetic speech detection," *Proc. Interspeech*, 2015.
6. X. Wu, R. He, Z. Sun and T. Tan, "A light CNN for deep face representation with noisy labels," *IEEE Trans. Information Forensics and Security*, vol. 13, no. 11, 2018.
7. G. Lavrentyeva et al., "STC antispoofing systems for the ASVspoof2019 challenge," *Proc. Interspeech*, 2019.
8. B. L. Welch, "The generalization of 'Student's' problem when several different population variances are involved," *Biometrika*, vol. 34, 1947.
9. Y. Benjamini and Y. Hochberg, "Controlling the false discovery rate: a practical and powerful approach to multiple testing," *J. Royal Statistical Society B*, vol. 57, no. 1, 1995.
10. J. Cohen, *Statistical Power Analysis for the Behavioral Sciences*, 2nd ed., Lawrence Erlbaum, 1988.
11. B. McFee et al., "librosa: Audio and music signal analysis in Python," *Proc. SciPy*, 2015.
12. F. Pedregosa et al., "Scikit-learn: Machine learning in Python," *JMLR*, vol. 12, 2011.
13. A. Paszke et al., "PyTorch: An imperative style, high-performance deep learning library," *NeurIPS*, 2019.